9.3.3 Configuring a request rate limiter

Depending on the requirements, you can configure the `RequestRateLimiter` filter for specific routes or as a default filter. In this case we'll configure it as a default filter so that it's applied to all routes, current and future.

The implementation of `RequestRateLimiter` on Redis is based on the *token bucket algorithm*. Each user is assigned a bucket inside which tokens are dripped over time at a specific rate (the *replenish rate*). Each bucket has a maximum capacity (the *burst capacity*). When a user makes a request, a token is removed from its bucket. When there are no more tokens left, the request is not permitted, and the user will have to wait until more tokens are dripped into its bucket.

NOTE If you want to know more about the token bucket algorithm, I recommend reading Paul Tarjan's "Scaling your API with Rate Limiters" article about how they use it to implement rate limiters at Stripe (https://stripe.com/blog/rate-limiters).

For this example, let's configure the algorithm so that each request costs `1` token (`redis-rate-limiter.requestedTokens`). Tokens are dripped in the bucket following the configured replenish rate (`redis-rate-limiter.replenishRate`), which we'll set as `10` tokens per second. Sometimes there might be spikes, resulting in a larger number of requests than usual. You can allow temporary bursts by defining a larger capacity for the bucket (`redis-rate-limiter.burstCapacity`), such as `20`. This means that when a spike occurs, up to 20 requests are allowed per second. Since the replenish rate is lower than the burst capacity, subsequent bursts are not allowed. If two spikes happen sequentially, only the first one will succeed, while the second will result in some requests being dropped with an `HTTP 429 -`